

Ser-FOX Attention Mask Equivalence

为什么当前 KV-cache 实现和原始 PI scorer 本质一致

Code-level alignment notes

2026-07-07

结论: same PI, same mask, incremental prefix cache.

主结论

语义

- 旧版 recompute 和当前 recompute 等价。
- 当前 KV-cache 和 recompute 在浮点误差内等价。
- candidate index isolation 未改变。

实现

- 旧版每步重算整个 prefix。
- 新版先缓存 prefix K/V。
- 每步只增量追加选中的 [index, value]。

验证

- old/current diff: **0**
- recompute/cache diff: **$4.77e-07$**
- CPU toy speedup: **1.25x**

PI 每一步做什么



关键不变量

候选 index token 共享同一个 logical frontier; 每个候选可以看完整 prefix, 但候选之间互相不可见。

$$prefix_{t+1} = prefix_t + [I_k, y_k]$$

原始 recompute 路径

```
for step in range(R):  
    idx_app = concat(prefix_t, all_index_tokens)  
    logits = score_parallel_indices(idx_app, R)  
    commit one [index, value]  
    prefix_t = concat(prefix_t, [index, value])
```

- 每一步重新 encode 正在增长的 prefix_t。
- 每一步也重新 encode 全部 candidate index block。
- prefix 每步增长 2 个 token: [index,value]。

Code: Ser-FOX/serfox_model.py:728 score_parallel_indices

当前 KV-cache 路径

```
cache = build_prefix_kv_cache(prefix_0)

for step in range(R):
    logits = score_parallel_indices_cached(all_index_tokens, cache)
    commit one [index, value]
    cache = append_to_kv_cache(cache, [index, value])
```

- prefix K/V 只构建一次。
- candidate block 仍然完整并行打分。
- 每步选中后，只把 committed [index,value] 追加进 cache。

Code: build_prefix_kv_cache:613, append_to_kv_cache:638, score_parallel_indices_cached:814

同一个 visibility mask

		keys / visible columns						
		p0	p1	p2	p3	l0	l1	l2
queries / rows	p0	1	0	0	0	0	0	0
	p1	1	1	0	0	0	0	0
	p2	1	1	1	0	0	0	0
	p3	1	1	1	1	0	0	0
	l0	1	1	1	1	1	0	0
	l1	1	1	1	1	0	1	0
	l2	1	1	1	1	0	0	1

Mask 规则

- prefix $\rightarrow$ prefix: 标准 causal lower triangle.
- candidate $i \rightarrow$ prefix: visible.
- candidate $i \rightarrow$ candidate i : visible.
- candidate $i \rightarrow$ candidate $j, i \neq j$: hidden.

```
visible = torch.eye(T)  
# used when causal_new_tokens=False
```

Code: build_ste_visible_mask:26, forward_new_tokens_with_kv:199

等价性计算

记 prefix cache 为:

$$K_p, V_p = \text{TransformerPrefix}(\text{prefix}_t)$$

candidate block 投影为:

$$Q_c, K_c, V_c = \text{Project}(\text{candidates})$$

对任意 candidate row, recompute 路径计算:

$$\text{softmax}([Q_c K_p^\top, Q_c K_c^\top \text{ masked-to-self}]) [V_p, V_c]$$

KV-cache 路径计算同一个式子:

$$\text{softmax}([Q_c K_p^\top, Q_c K_c^\top \text{ masked-to-self}]) [V_p, V_c]$$

因此, 令

$$S_{\text{full}} = \text{score_parallel_indices}(\cdot), \quad S_{\text{cache}} = \text{score_parallel_indices_cached}(\cdot),$$

则在同一 prefix 和 candidate block 下:

$$S_{\text{full}} \equiv S_{\text{cache}}.$$

速度提升来自哪里

旧版

1. recompute prefix_t + all candidates
2. commit [index,value]
3. repeat with longer prefix

当前

1. build prefix K/V once
2. score candidates against cache
3. append only [index,value]

正确表述

KV-cache 加速的是 prefix accumulation, 避免重复 encode 已经确定的 serialized prefix; PI candidate scoring 仍然是同一个并行打分问题。

简单数值验证

Setup

- CPU only
- B=16
- 3 layers, 4 heads
- embd=128
- quiz=16, response=36

Old vs current

- max_abs_diff: **0**
- finite masks equal: **True**

Recompute vs cache

- max_abs_diff: **4.77e-07**
- finite masks equal: **True**

解释：cache 路径和 recompute 路径只有正常浮点误差量级差异。

简单速度对比

Path	Median time	Relative
old recompute	1166.1 ms	1.00x
current recompute	1168.0 ms	1.00x
current KV-cache	937.7 ms	1.25x faster

- current recompute 和 old recompute 基本同速。
- 加速来自 KV-cache 去掉重复 prefix encode。
- CPU toy benchmark 只作为方向性说明；正式 GPU/大任务需要单独 benchmark。

改了什么，没有改什么

Changed

- `build_prefix_kv_cache`
- `append_to_kv_cache`
- `score_parallel_indices_cached`
- `generation / regeneration` can use cache path

Did not change

- single-index visibility rule
- candidate index isolation
- PI position-choice semantics
- candidate value logits surface

推荐对外说法

Compact explanation

Our current code is an incremental implementation of the same PI scorer. The original code recomputed the entire serialized prefix plus all candidate index tokens at every step. The new code caches the prefix K/V once, scores the same candidate index block against that cache using the same index-isolating visibility mask, and after selecting one position appends only the committed [index,value] pair. Therefore the mathematical scorer is the same; the speedup comes from eliminating repeated prefix computation.

Caveats

`use_rope=True` and grouped-index scoring are new optional paths. `norepeat` / `pad_eos_last` are decode-time logit masks, not attention masks.